# Лабораторна робота №3

Функціональне програмування

Аналіз речень навчального тексту

Левківський Євгеній Сергійович

ІПС-41

## 1. Мета роботи

Створити програму обробки тексту підручника з програмування на Haskell. Використати окремі типи для символу, слова, речення й розділового знака. Забезпечити читання з файла та нормалізацію пробільних символів.

## 2. Постановка завдання

Обрано перше завдання: знайти найбільшу кількість речень тексту, в яких є однакові слова. Для кожного слова потрібно визначити кількість різних речень, де воно зустрічається, а потім знайти найбільше з цих значень. Додатково виводяться всі слова, що досягли максимуму, та номери відповідних речень.

Табуляції, повторювані пробіли та перенесення рядків у сформованому тексті замінюються одним пробілом. Текст зберігається у UTF-8; на початку допускається BOM. Навчальний приклад про Haskell наведено у lab3/examples/textbook.txt.

## 3. Модель тексту

Використано типи Character, WordToken, Punctuation, Element, Sentence та Document. Character є обгорткою над Char, WordToken — над Text, Punctuation — над Character. Element розрізняє слово й розділовий знак. Sentence зберігає нормалізований текст і список елементів, Document — список речень.

```
newtype Character = Character Char
newtype WordToken = WordToken Text
newtype Punctuation = Punctuation Character
data Element = WordElement WordToken
             | PunctuationElement Punctuation
data Sentence = Sentence
  { sentenceText :: Text, elements :: [Element] }
newtype Document = Document [Sentence]
```

Ключові чисті функції: normalizeSpaces, parseDocument, sentenceWords, analyze, renderAnalysis. Введення-виведення відокремлено в Main.hs, що дає змогу перевіряти логіку без файлової системи.

## 4. Правила розбору

Функція T.unwords . T.words зводить пробільні послідовності до одного пробілу. Речення закінчуються крапкою, знаком питання чи оклику. Послідовність ?! або ... формує одну межу; завершальні лапки й дужки приєднуються до речення. Крапка між цифрами в числі 3.14 не є межею. Останній фрагмент без кінцевого знака також враховується.

Слово починається з літери Unicode. Усередині допускаються літери, а також апостроф або дефіс між літерами. Слова нормалізуються через toCaseFold; апострофи ’ та ’ замінюються звичайним апострофом. Завдяки цьому «Пам’ять» і «пам'ять» вважаються однаковим словом.

Граматичні скорочення, морфологічні форми та лематизація не аналізуються. Наприклад, «функція» та «функцію» є різними словами, а скорочення «т.д.» розбивається за крапками. Ці правила явно визначають межі алгоритму.

## 5. Алгоритм пошуку

Речення нумеруються від 1. Для кожного речення список слів перетворюється на Set, що усуває повторення всередині речення. Далі словник Map WordToken [Int] накопичує номери речень для кожного слова. Обчислюється найбільша довжина списку; усі записи з такою довжиною потрапляють до результату й виводяться за порядком ключів.

Принципова відмінність від підрахунку входжень: речення «Код код код!» містить слово «код» тричі, але дає лише один внесок. Для тексту «Код код код! Код працює. Новий код?» результат дорівнює 3, а не 5.

Для порожнього тексту або тексту без слів максимум дорівнює 0. Якщо слова є, але між реченнями немає повторень, максимум дорівнює 1. За однакового максимуму виводяться всі відповідні слова.

Токенізація має лінійну складність за кількістю символів. З урахуванням множин і словника аналіз займає O(W log V), де W — кількість слів, V — кількість різних слів. Пам’ять потрібна для тексту та індексу входжень.

## 6. Запуск і отриманий результат

```
lab3 lab3/examples/textbook.txt
lab3 lab3/examples/textbook.txt --output lab3/results.txt
```

На підготовленому Windows-комп’ютері виконуваний файл розміщено в .build/lab3.exe. Програма показує результат у консолі та за потреби записує його як UTF-8 файл. Помилковий UTF-8 або недоступний файл дає зрозуміле повідомлення й ненульовий код завершення.

Фактичний вивід для навчального тексту:

```
Речень: 11
Найбільша кількість речень зі спільним словом: 5
значення — речення 2, 4, 7, 10, 11
```

У другому реченні слово «значення» трапляється двічі, але рахується один раз. Речення «Значення 3.14 є числовим літералом.» залишається одним реченням, оскільки крапка між цифрами не завершує його. Повний нормалізований текст та відповідь збережено в lab3/results.txt і verification/lab3.txt.

## 7. Перевірки

| Вхідні умови | Очікувана поведінка | Результат |
|---|---|---|
| Порожній текст або самі пробіли | Максимум 0 | Підтверджено |
| Повторення слова в одному реченні | Один внесок на речення | Підтверджено |
| Різний регістр HASKELL / Haskell | Однакове слово | Підтверджено |
| Різні апострофи в українських словах | Однакова нормалізована форма | Підтверджено |
| Табуляції та перенесення рядків | Один пробіл | Підтверджено |
| Число 3.14 та група ?! | Правильні межі речень | Підтверджено |
| Речення без кінцевого знака | Фрагмент враховано | Підтверджено |
| Декілька слів з однаковим максимумом | Виведено всі | Підтверджено |

Тести виконано GHC 9.6.7. Сукупний набір для трьох робіт містить 42 перевірки; усі пройшли. Окремий інтеграційний запуск перевірив читання прикладу з диска та збереження результату.

## 8. Висновок

Реалізовано програму аналізу навчального тексту з явною моделлю символів, слів, розділових знаків і речень. Враховано Unicode, українські апострофи, нормалізацію пробілів та повторення слова всередині речення. Для прикладу з 11 речень встановлено максимум 5 для слова «значення».

## 9. Джерела

GHC User’s Guide: https://downloads.haskell.org/ghc/9.6.7/docs/users_guide/

Haskell text та containers: https://hackage.haskell.org/